

CSCE 605 COMPILER DESIGN · MODULE 06 · LECTURE NOTES

Type Checking

Proving, before it runs, that the operations make sense.

MODULE OUTCOMES

- Implement a type checker for a small language.
- Explain soundness and completeness, and why both are impossible.
- Implement Hindley–Milner inference and state its limits.
- Explain subtyping and variance.
- Explain how shader languages differ and why.

1 What a type system is for

AN AUTOMATED PROOF OF A DELIBERATELY WEAK THEOREM

A **type system proves, before execution, that a certain class of error cannot occur** — typically that no operation is ever applied to a value it cannot handle. That is a real theorem about the program, mechanically established.

The theorem is deliberately weak, because the proof must be found automatically, for every program, fast enough to run on every compile. Stronger theorems exist and cost more — which is the entire design space, from dynamic typing through to dependent types and full verification.

Rice's theorem is the fundamental constraint. Every non-trivial semantic property of programs is undecidable (CSCE 627), so **a type checker must be incomplete, or unsound, or non-terminating** — there is no fourth option, and no amount of cleverness produces one.

Nearly every system chooses incomplete: it rejects some programs that would in fact have run without error. **That rejection is the price, and the whole design question is what you buy with it** — how many real errors are caught per correct program wrongly rejected, and how annoying the rejections are to work around.

Property	What it means	Who gives it up, and why
Sound	If the checker accepts it, no type error occurs at runtime.	C and C++ (casts, unions, undefined behaviour); Java (array covariance, §3); TypeScript deliberately, to stay usable with existing JavaScript idioms — and it documents the unsoundness rather than hiding it.
Complete	If no type error would occur, the checker accepts it.	Essentially every type system. This is the one that Rice's theorem takes, and giving it up is the standard choice.

Property	What it means	Who gives it up, and why
Decidable	The checker always terminates.	C++ template instantiation (Turing-complete, and the standard only suggests a recursion limit); full dependent types without restrictions.
Inferring	Type annotations are optional and are reconstructed.	C and older Java require annotations nearly everywhere; Go and modern Java infer locally but not across function boundaries — a deliberate choice, since full inference makes error messages much worse.

2 Checking

```

G |- e1 : int      G |- e2 : int
-----
G |- e1 + e2 : int      (T-ADD)

G |- c : bool      G |- a : T      G |- b : T
-----
G |- if c then a else b : T      (T-IF)

```

"G |- e : T" reads: in environment G, expression e has type T.

```

Type check(Env& G, Expr* e) {
    switch (e->kind) {
    case ADD: {
        Type l = check(G, e->lhs), r = check(G, e->rhs);
        if (l != INT || r != INT)
            error(e->span, "operands of + must be int");
        return INT;
    }
    ...
    }
}

```

Inference rules are not decoration. Each rule becomes one case of one recursive function: the premises are the recursive calls, the side conditions are the checks, and the conclusion is the return value. **Reading a rule bottom-up gives you the code.** Note also that T-IF requires both branches to have the *same* type — **that is a language design decision, not a necessity**, and relaxing it to 'some common supertype' is exactly what requires the subtyping machinery of §3.

3 Inference and subtyping

HINDLEY–MILNER: UNIFICATION OVER TYPE VARIABLES

Assign every unknown type a fresh type variable. Then walk the program generating *constraints* — 'the type of this argument must equal the type of that parameter', 'the condition must be bool'.

Solve the constraint set by unification, the same algorithm used in logic programming: repeatedly match structures and bind variables, **with an occurs check** to reject constraints like $\alpha = \alpha \rightarrow \alpha$ that would require an infinite type.

Generalise at let bindings, quantifying over the type variables not free in the environment. **This is what produces polymorphism**: a function bound by let and used at two different types gets two instantiations of one general scheme.

The result infers principal types — the most general type each expression can have — **with no annotations whatsoever**, which is a genuinely remarkable result and is why ML and Haskell feel the way they do. **Adding subtyping, or higher-rank polymorphism, breaks it**: principal types stop existing, and inference becomes undecidable. That is why most mainstream languages infer locally and require annotations at function boundaries — a deliberate retreat that also makes the error messages dramatically better.

VARIANCE IS WHERE SUBTYPING GETS SUBTLE

If Cat is a subtype of Animal, is List<Cat> a subtype of List<Animal>? The intuitive answer is yes and the intuitive answer is wrong.

It is sound only if the list is read-only. A mutable List<Animal> accepts a Dog; so if a List<Cat> could be used as one, you could insert a Dog into a list of Cats and the next read would return a Dog where a Cat was guaranteed.

The general rule: covariant in output positions, contravariant in input positions. A function returning Cat is usable wherever one returning Animal is wanted (outputs are covariant); a function *taking* Animal is usable wherever one taking Cat is wanted (inputs are contravariant — it accepts more, so it is more general). **Mutable containers are both, so they must be invariant.**

Java made arrays covariant and they are therefore unsound. The language compensates with a runtime check on every array store — an ArrayStoreException that exists purely to patch a type system hole. **A design decision from 1995 that every Java program has paid for on every array write ever since**, and the clearest illustration available that soundness is worth getting right the first time.

4 Shader type systems

Feature	In shading languages	Why
Pointers	Absent, or heavily restricted to specific storage classes.	No aliasing means far better optimisation — see the callout.
Recursion	Forbidden in most shader stages.	There is no call stack. Register allocation must be statically bounded across thousands of simultaneous invocations.
Vector types	First-class: float4, swizzles like .xyzw and .rgba, component-wise operators.	The hardware is SIMD and the natural data is 2-to-4 component, so the type system matches the machine directly rather than relying on auto-vectorisation (CSCE 735 Module 02).
Precision qualifiers	half, lowp, mediump, highp as part of the type.	Mobile power and bandwidth. Half precision halves the register pressure and the memory traffic, and for colour it is usually sufficient.
Uniform versus varying	Encoded in the type system (or in the storage class).	It decides where a value physically lives — a constant buffer, an interpolated attribute, or a register — which the compiler must know statically.
Dynamic allocation	None at all.	Thousands of concurrent invocations and no allocator. All storage is statically sized.

THE RESTRICTIONS ARE WHAT MAKE THE OPTIMISATION POSSIBLE

A C compiler usually cannot prove that two pointers do not alias, so it must assume they might — which blocks reordering, blocks keeping a value in a register across a store, and blocks vectorisation, constantly and invisibly. restrict exists because this is the dominant obstacle to optimising C.

A shader has no pointers, so the provenance of every value is known exactly and aggressive transformation is always legal. The compiler never has to be conservative about memory.

No recursion means no call stack, so every call can be inlined, and register pressure is a statically computable quantity rather than a dynamic one (Module 11). No allocation means no garbage collector and no allocation-related side effects to order around.

So shader compilers routinely achieve transformations that C compilers cannot — not because they are better engineered, but because the language handed them a far stronger guarantee. Restricting the language is how you buy optimisation, and that trade is visible everywhere: Fortran beats C on numerics for the aliasing reason, Rust's borrow checker enables noalias by construction, and pure functional languages can reorder almost anything.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Pierce — Types and Programming Languages	The standard reference. Chapters on simple types, subtyping, and variance cover §1 through §3. Library copy; the author's slides are free.
Stanford CS143 — type checking lectures (free)	The rule-to-code correspondence of §2, worked through.
Damas & Milner; and 'Write You a Haskell' (free)	Hindley–Milner implemented , with unification and generalisation in readable code.
Microsoft — HLSL reference; Khronos — GLSL specification (free)	The §4 restrictions, stated normatively. The storage-class and precision sections are the interesting ones.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write the typing rules for your project language on one page.
- 2 Implement the checker directly from those rules, one case each.
- 3 **Produce a type error message with both spans** — where the expected type came from and where the actual one did.
- 4 Find a program your checker rejects that would have run correctly. **That is incompleteness; describe it precisely.**
- 5 Implement unification with an occurs check.
- 6 **Implement Hindley–Milner** for a small functional language and verify it infers principal types.
- 7 Construct a program where HM infers a type you did not expect, and explain it.
- 8 **Demonstrate Java's array covariance hole** and the runtime exception it produces.
- 9 Implement variance checking for a generic container.
- 10 Write a shader, then write the equivalent C, and compare the generated code. **Explain the difference using aliasing.**

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What theorem does a type system prove, and why is it weak?
- 2 State Rice's theorem's consequence for type checking.
- 3 Define sound and complete, and say who gives up which.

- 4 How does an inference rule become code?
- 5 Describe Hindley–Milner and say what breaks it.
- 6 State the variance rule and explain Java's array hole.
- 7 Name six ways shader type systems differ from C.
- 8 Why do those restrictions enable better optimisation?